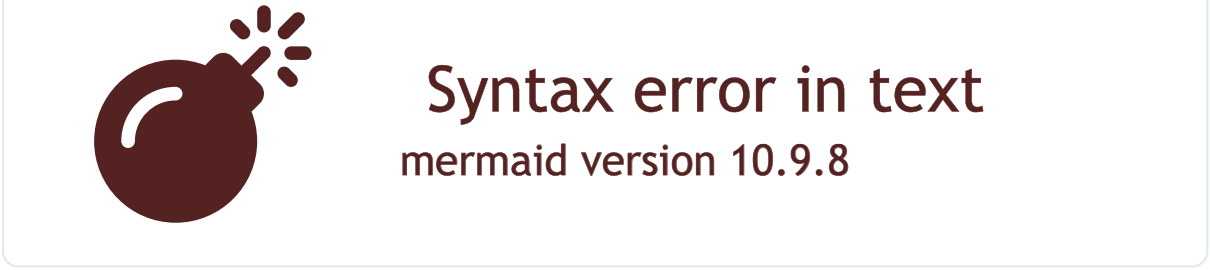
Lecture 21.08: Vector Addition

📌 Topic Overview

In this lecture, Shraddha ma'am demonstrates **Vector Addition**, working through her blackboard example with vectors $\vec{A} = (2, 1, 3)$ and $\vec{B} = (4, 5, 2)$ to illustrate component-wise summation, the Triangle Law of Addition, and the Parallelogram Law.

🏛️ Parallelogram Law of Vector Addition



📐 Mathematical Formulation: Component-Wise Addition

Given two vectors in $\mathbb{R}^3$:

$$\vec{A} = a_1\hat{i} + b_1\hat{j} + c_1\hat{k}$$

$$\vec{B} = a_2\hat{i} + b_2\hat{j} + c_2\hat{k}$$

Their resultant sum $\vec{R} = \vec{A} + \vec{B}$ is computed by summing corresponding spatial components:

$$\vec{R} = (a_1 + a_2)\hat{i} + (b_1 + b_2)\hat{j} + (c_1 + c_2)\hat{k}$$

Blackboard Numerical Example:

- $\vec{A} = 2\hat{i} + \hat{j} + 3\hat{k}$
- $\vec{B} = 4\hat{i} + 5\hat{j} + 2\hat{k}$

$$\vec{R} = (2+4)\hat{i} + (1+5)\hat{j} + (3+2)\hat{k} = 6\hat{i} + 6\hat{j} + 5\hat{k} = (6, 6, 5)$$

Resultant Magnitude:

$$\|\vec{R}\| = \sqrt{6^2 + 6^2 + 5^2} = \sqrt{36 + 36 + 25} = \sqrt{97} \approx 9.8489$$

💻 Python Implementation

```
import numpy as np

def add_vectors_componentwise(vec_a=(2, 1, 3), vec_b=(4, 5, 2)):
    """
    Performs component-wise vector addition for two vectors A and B:
    R = A + B = (a1 + b1)i + (a2 + b2)j + (a3 + b3)k
    Demonstrates both analytical component math and NumPy vectorization.

    Parameters:
        vec_a (tuple): First vector components.
        vec_b (tuple): Second vector components.

    Returns:
        dict: Components of A, B, and the resultant vector R.
    """
    a_arr = np.array(vec_a)
    b_arr = np.array(vec_b)

    # Vectorized element-wise addition
    resultant = a_arr + b_arr

    return {
        "vector_A": list(vec_a),
        "vector_B": list(vec_b),
        "resultant_R": list(resultant),
        "R_notation_str": f"{resultant[0]}i + {resultant[1]}j + {resultant[2]}k",
        "magnitude_R": float(np.linalg.norm(resultant))
    }

# --- Execution & Verification ---
# Shraddha ma'am's exact chalkboard example: A = (2, 1, 3), B = (4, 5, 2)
res = add_vectors_componentwise(vec_a=(2, 1, 3), vec_b=(4, 5, 2))
print("Vector A:", res["vector_A"], "-> 2i + 1j + 3k")
print("Vector B:", res["vector_B"], "-> 4i + 5j + 2k")
print("Resultant R = A + B:", res["resultant_R"])
print(f"Component Form: {res['R_notation_str']}")
print(f"Resultant Magnitude: ||R|| = {res['magnitude_R']:.4f}")
```

🔑 Key Takeaways

1. Vector addition is strictly component-wise: add $\hat{i}$ with $\hat{i}$, $\hat{j}$ with $\hat{j}$, and $\hat{k}$ with $\hat{k}$.
2. Geometrically, vector addition connects the tail of the second vector to the tip of the first vector (Triangle Law).

3. In deep neural networks, residual skip connections (ResNet) operate via direct element-wise vector addition: $\mathbf{x}_{\text{out}} = \mathcal{F}(\mathbf{x}) + \mathbf{x}$.